KPIT | APEX LAB

POSIX Pipes

Inter-Process Communication (IPC) using Anonymous Pipes

Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What Is a Pipe?

A unidirectional communication channel between processes

- Data flows **like a queue**: first-in, first-out (FIFO)
- Created using **pipe() system call**
- Exists in memory (**kernel buffer**)
- Commonly used between **parent-child processes**

Anonymous Pipes Features

- Not associated with a file name
- Only work between related processes(parent-child or siblings created via fork())
- Automatically destroyed when both ends close
- Provide half-duplex communication (one-way)

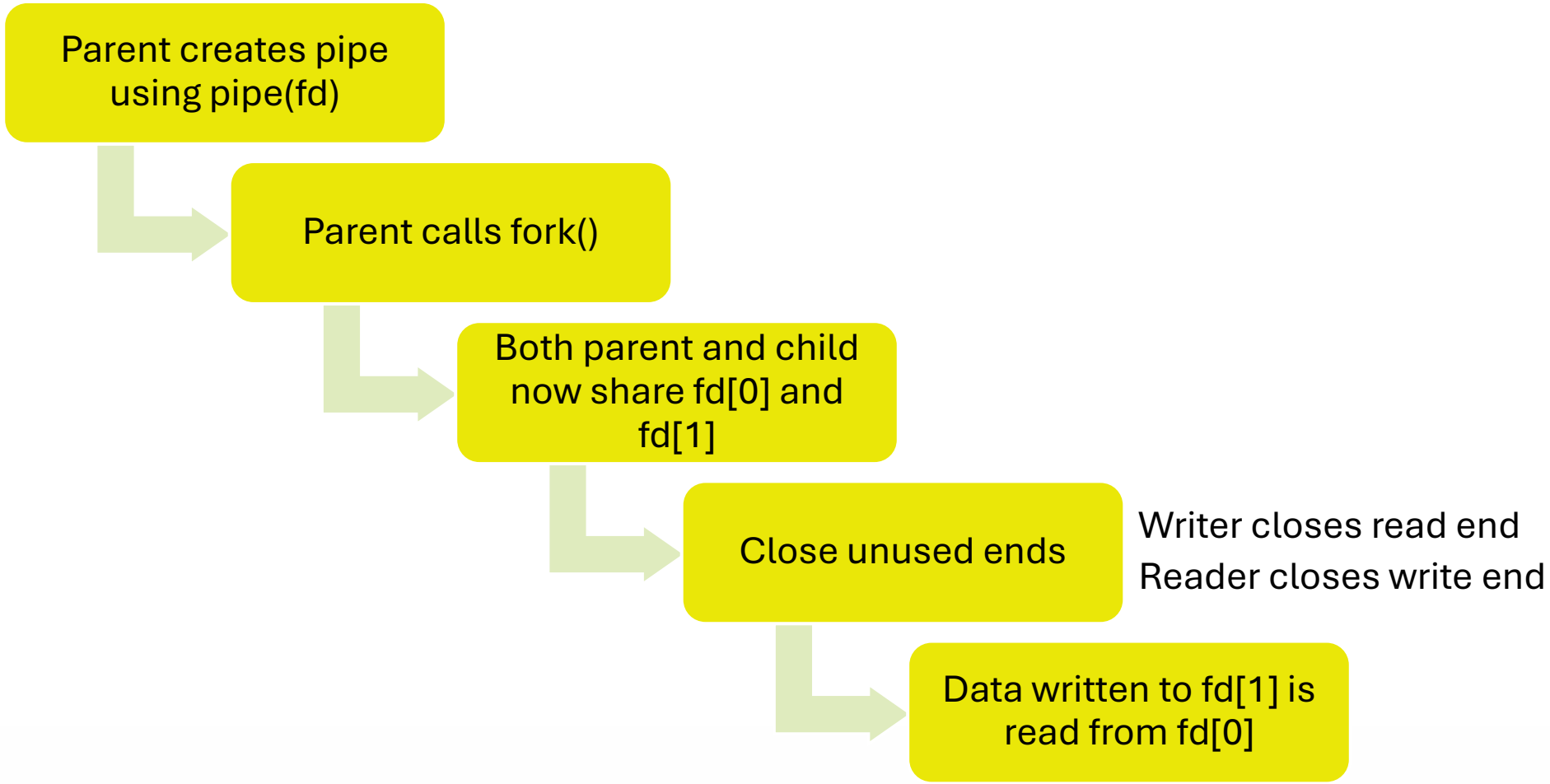
Anonymous pipes enable simple, fast IPC within a family of processes.

pipe() System Call

- int pipe(int fd[2]);
- Returns:
  - fd[0]: Read end
  - fd[1]: Write end
- On success: returns 0
- On failure: returns -1, sets errno

- Common errors:
  - EMFILE: Too many file descriptors
  - ENFILE: System FD limit reached

Pipe Lifecycle



Example 1: Parent Writes, Child Reads (1/2)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>

int main(void)
{
int pipefd[2]; // pipefd[0] = read-end, pipefd[1] =
write-end
pid_t pid;
const char *msg = "Hello from parent!\n";

if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
}
```

Example 1: (2/2)

```
pid = fork();
if (pid == 0) { // ---------- CHILD ----------
        close(pipefd[1]); // close unused write end

        char buffer[128];
        ssize_t nbytes = read(pipefd[0], buffer,
        sizeof(buffer));
        if (nbytes == -1) {
                perror("read");
                exit(EXIT_FAILURE);
        }
        buffer[nbytes] = '\0';
        // NUL-terminate for safe printing

        printf("Child received: %s", buffer);
        close(pipefd[0]); // done reading
        _exit(EXIT_SUCCESS);
}
```

```
else { // ---------- PARENT ----------
        close(pipefd[0]); // close unused read end

        ssize_t len = strlen(msg);
                if (write(pipefd[1], msg, len) != len) {
                perror("write");
                exit(EXIT_FAILURE);
        }
        close(pipefd[1]); // signal EOF to child

        int status;
        waitpid(pid, &status, 0); // optional: reap child
        if (WIFEXITED(status))
                printf("Child exited with status %d\n",
        WEXITSTATUS(status));
        else
                 printf("Child terminated abnormally");
}

return 0;
}
```

Example 2: Child Writes, Parent Reads (1/2)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>

int main(void)
{
int pipefd[2]; // pipefd[0] = read end, pipefd[1] = write end
pid_t pid;
const char *msg = "Hello from child!\n";

/* Create the pipe */
if (pipe(pipefd) == -1) {
perror("pipe");
exit(EXIT_FAILURE);
}
```

Example 2: (2/2)

```
pid = fork();

if (pid == 0) { // ---------- CHILD ----------
close(pipefd[0]); // child does not read -> close
read end

size_t len = strlen(msg);
if (write(pipefd[1], msg, len) != (ssize_t)len) {
perror("write");
exit(EXIT_FAILURE);
}

close(pipefd[1]); // send EOF to parent
_exit(EXIT_SUCCESS);
}
```

```
else { // ---------- PARENT ----------
        close(pipefd[1]); // parent does not write -> close write end

        char buffer[128];
        ssize_t n = read(pipefd[0], buffer, sizeof(buffer));
        if (n == -1) {
                perror("read");
                exit(EXIT_FAILURE);
        }
        buffer[n] = '\0'; // NUL-terminate for safe printing

        printf("Parent received: %s", buffer);
        close(pipefd[0]); // done reading

        int status;
        waitpid(pid, &status, 0); // reap child
        if (WIFEXITED(status))
                printf("Child exited with status %d\n",
                WEXITSTATUS(status));
        else
                printf("Child terminated abnormally");
}
return 0;
}
```

Two-way Communication Using Two Pipes (1/3)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MSGSIZE 128

int main(void)
{
	int pipe_parent_to_child[2];
	// Parent writes, child reads
	int pipe_child_to_parent[2];
	// Child writes, parent reads
```

```c
/* Create both pipes */
if (pipe(pipe_parent_to_child) == -1 ||
pipe(pipe_child_to_parent) == -1) {
	perror("pipe");
	exit(EXIT_FAILURE);
}
```

Two-way Communication Using Two Pipes (2/3)

```
pid_t pid;
pid = fork();
if (pid == -1) {
perror("fork");
exit(EXIT_FAILURE);
}

/* ------------- CHILD PROCESS ------------- */
if (pid == 0) {
        /* Close unused ends */
        close(pipe_parent_to_child[1]);
        // close write end (parent→child)
        close(pipe_child_to_parent[0]);
        // close read end (child→parent)
```

```
/* Read a message from the parent */
char buf[MSGSIZE] = {0};
read(pipe_parent_to_child[0], buf, MSGSIZE - 1);
printf("Child received: \"%s\"\n", buf);

/* Prepare a reply */
const char *reply = "Hi parent, message received!";
write(pipe_child_to_parent[1], reply, strlen(reply) + 1);
/* Clean up */
close(pipe_parent_to_child[0]);
close(pipe_child_to_parent[1]);
_exit(EXIT_SUCCESS);
}
```

Two-way Communication Using Two Pipes (3/3)

```
/* ------------ PARENT PROCESS ------------ */
else {
/* Close unused ends */
close(pipe_parent_to_child[0]);
// close read end (parent→child)
close(pipe_child_to_parent[1]);
// close write end (child→parent)

/* Send a message to the child */
const char *msg = "Hello child, this is parent.";
write(pipe_parent_to_child[1], msg, strlen(msg) + 1);
/* Receive child's reply */
char reply_buf[MSGSIZE] = {0};
read(pipe_child_to_parent[0], reply_buf, MSGSIZE - 1);
printf("Parent received: \"%s\"\n", reply_buf);
```

```
/* Clean up */
close(pipe_parent_to_child[1]);
close(pipe_child_to_parent[0]);

/* Wait for child to exit */
wait(NULL);
}

return 0;
}
```

Blocking Behavior of Pipes

When read blocks:	When write blocks:	When read returns 0:	When write fails with SIGPIPE:
• Pipe empty • Write end still open	• Pipe buffer full	• All write ends closed → EOF	• All read ends closed → writer gets SIGPIPE

Pipe Buffering

- Pipes use kernel buffer (size ≈ 64 KB on modern Linux)
- Writes ≤ PIPE_BUF are atomic
- Good for logging or sending structured messages

Closing Pipe Ends Correctly

Golden Rules:

- Close **unused ends immediately** after fork
- Writers must close when done → allows readers to detect EOF
- Readers must close when done → prevents writers from blocking forever

Incorrect closure → deadlocks & hangs

Pipes in Embedded Systems

Typical use cases:

- Logging process receiving data via a pipe
- Sensor poller → filter thread (as separate process)
- Motor control command-forwarder
- Inter-process pipelines without using shared memory

Troubleshooting & Common Errors

- Deadlock due to not closing unused ends
- Reader blocked forever (writer didn't close)
- Writer SIGPIPE (reader closed early)
- Reading partial messages without framing
- Forgetting to flush stdout when writing from parent/child

Summary

- Pipes: simplest form of IPC
- Unidirectional & anonymous
- Use with fork() for parent-child communication
- Understand blocking, EOF, atomic writes
- Use two pipes for duplex communication

Scenario: Parent–Child Line Filter with Two-Way Control (1/4)

Context

- Build a line filter as a pair of related processes:
- The parent streams input lines to the child via a pipe;
- The child transforms lines (e.g., uppercase or ROT13) and sends results back to the parent through a second pipe.
- Parent prints transformed lines and supports a runtime mode switch command.

Scenario: Parent–Child Line Filter with Two-Way Control (2/4)

PART 1 — Plumbing & Direction

- Create two pipes:
  - p2c: parent → child (write at parent, read at child)
  - c2p: child → parent (write at child, read at parent)
- fork(); close unused ends properly in each process.

Scenario: Parent–Child Line Filter with Two-Way Control (3/4)

PART 2 — Transform Logic (Child)

- Default transform: uppercase.
- Read a line from p2c, transform, write to c2p.
- Exit on EOF when parent closes its write end.

Scenario: Parent–Child Line Filter with Two-Way Control (4/4)

PART 3 — Control Channel (Parent)

- Parent reads stdin:
  - Normal lines → send to child, read transformed reply, print.
  - Command :mode rot13 → send control token __CMD_MODE_ROT13__ to child; child switches mode.
  - Command :quit → close p2c write end; read remaining output; exit.